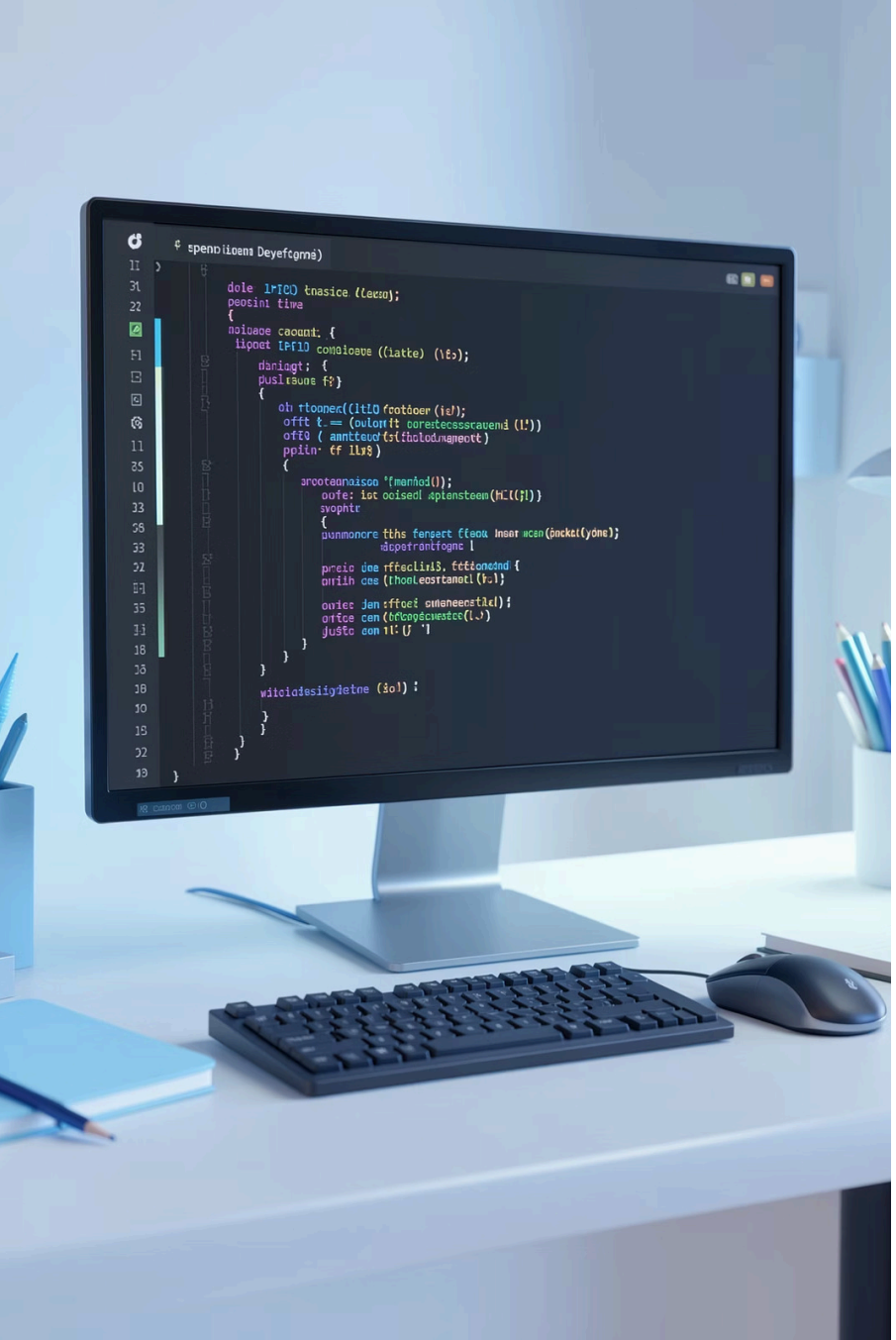
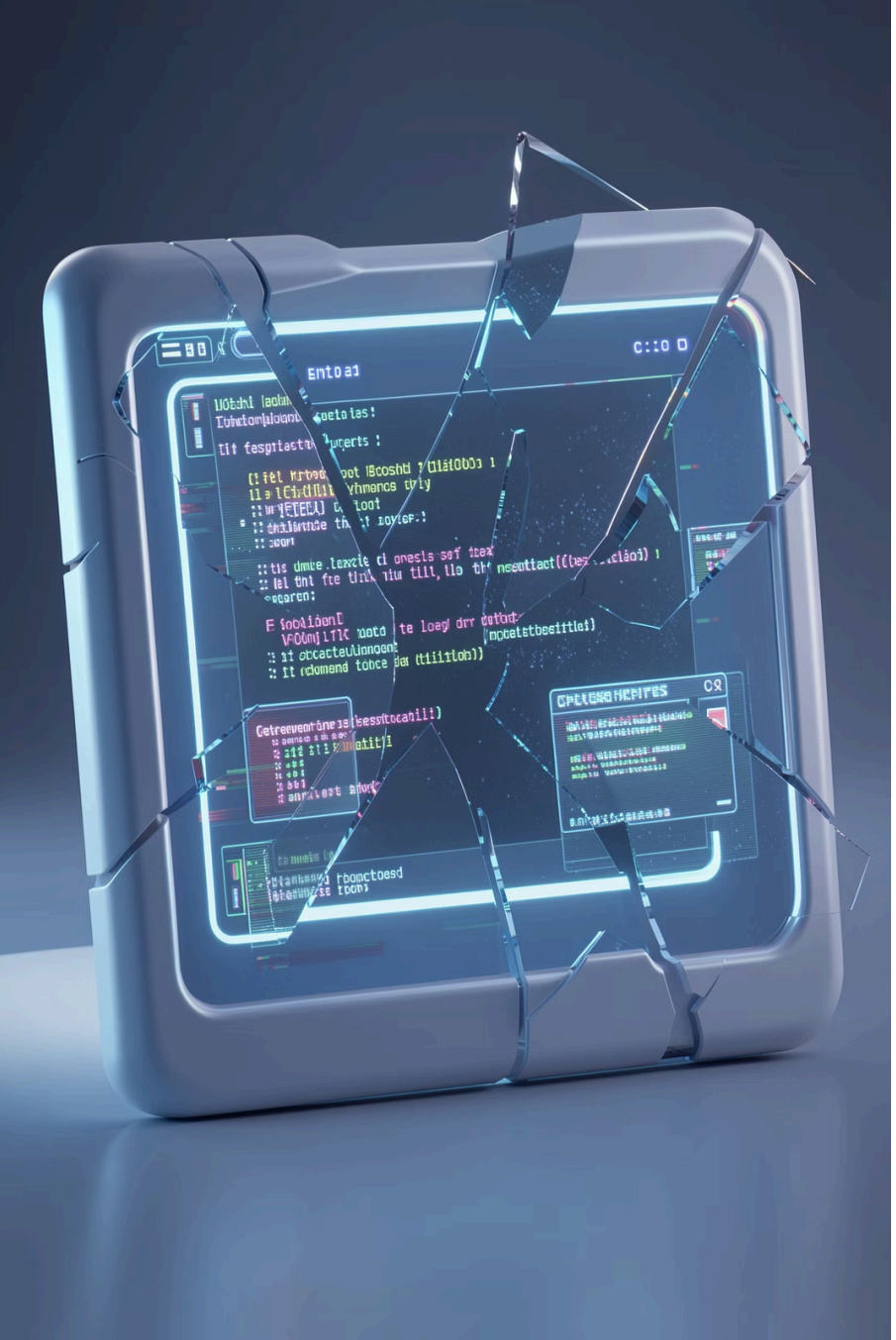


Java: Archivos y Excepciones en la práctica

De programas que olvidan todo a aplicaciones que persisten datos

Una guía de trabajo paso a paso para construir miniproyectos en consola que guardan información, la recuperan y manejan errores de forma controlada.



El problema inicial

> "Tu programa funciona... hasta que lo cierras"

Sin persistencia

Cada vez que el programa termina, todos los datos introducidos desaparecen por completo.

Sin memoria

No hay forma de recuperar lo que el usuario introdujo en una sesión anterior.

Sin control de errores

Cualquier entrada inesperada o fallo interno detiene el programa con una excepción no controlada.

Objetivo del recorrido

Este recorrido tiene un propósito concreto: transformar programas frágiles y olvidadizos en aplicaciones que funcionan de verdad.

Guardar datos en archivos

Hacer que la información sobreviva al cierre del programa.

Leer datos persistidos

Recuperar lo guardado en cualquier momento posterior.

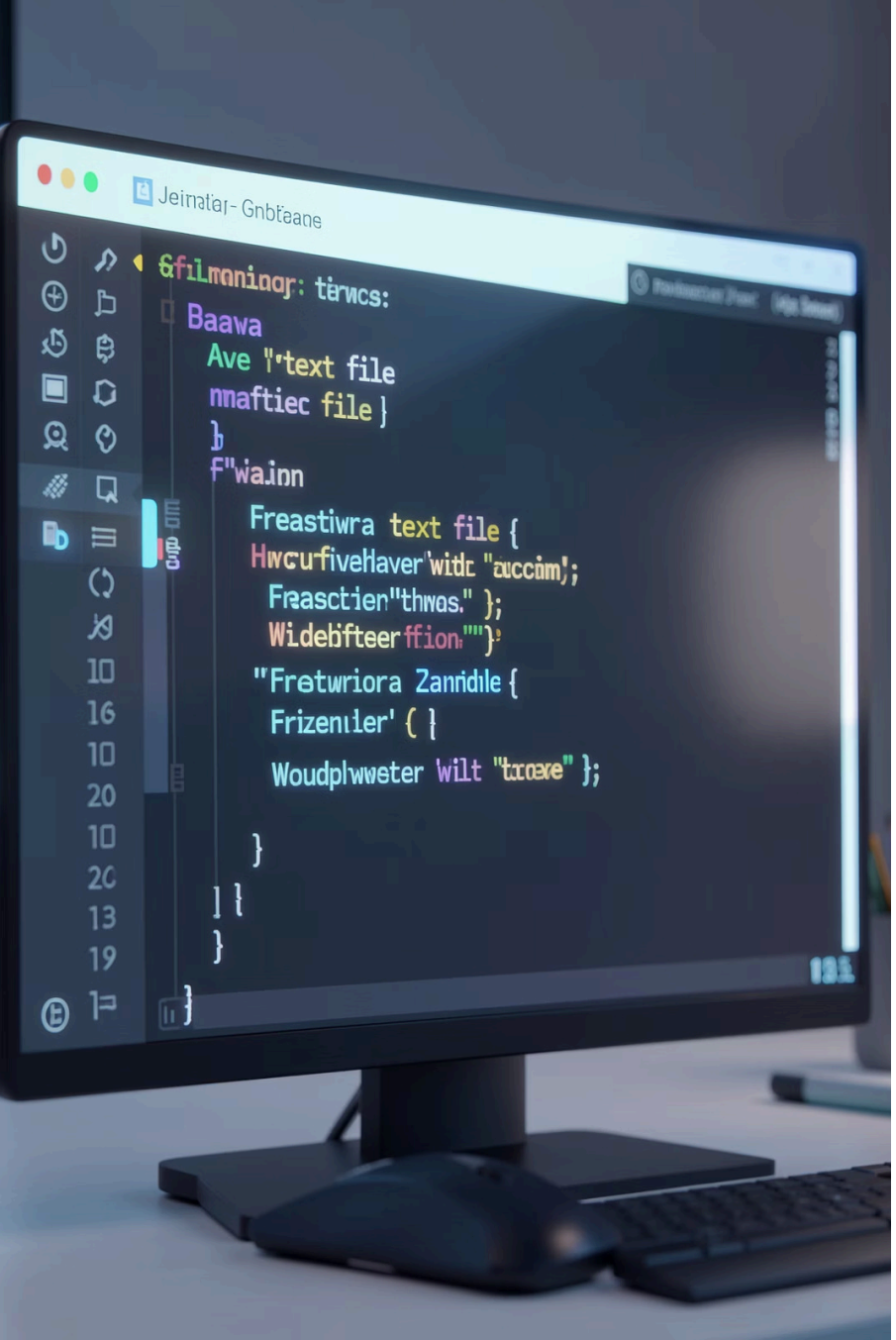
Manejar errores sin fallar

Anticipar problemas para que el programa responda con elegancia en lugar de detenerse.

Mapa del recorrido

Ocho miniproyectos, cada uno con un objetivo claro. Cada paso construye sobre el anterior.

1. **Crear archivo** — Primera escritura en disco
2. **Append** — Acumular sin sobrescribir
3. **Leer archivo** — Línea por línea
4. **Sistema de ventas** — Integración completa
5. **Excepciones** — Errores controlados
6. **Try-with-resources** — Cierre automático
7. **Archivo inexistente** — Tipos de error
8. **Programa robusto** — Validación completa



MINIPROYECTO 1

Crear un archivo de texto

> "El programa deja evidencia en disco"

Qué se usa

- **FileWriter** — abre o crea el archivo
- **BufferedWriter** — escribe de forma eficiente
- **write()** y **newLine()** — insertan contenido

Qué se logra

Por primera vez, el programa genera un fichero real en el sistema de archivos. Aunque el programa se cierre, el archivo permanece. Es la primera forma de persistencia real en Java.

MINIPROYECTO 2

Guardar datos con Append

> "Ahora los datos se acumulan, no se pierden"

El modo **append** es la diferencia entre sobrescribir y acumular. Sin él, cada ejecución destruye lo anterior.

Sin append

Cada vez que se escribe, el archivo se borra y se reemplaza desde cero. Los registros anteriores desaparecen.

Con append

`new FileWriter("datos.txt", true)` — el segundo parámetro activa el modo acumulación. Los nuevos datos se añaden al final.

MINIPROYECTO 3

Leer un archivo línea por línea

> "Guardar no sirve sin poder leer"

FileReader + BufferedReader

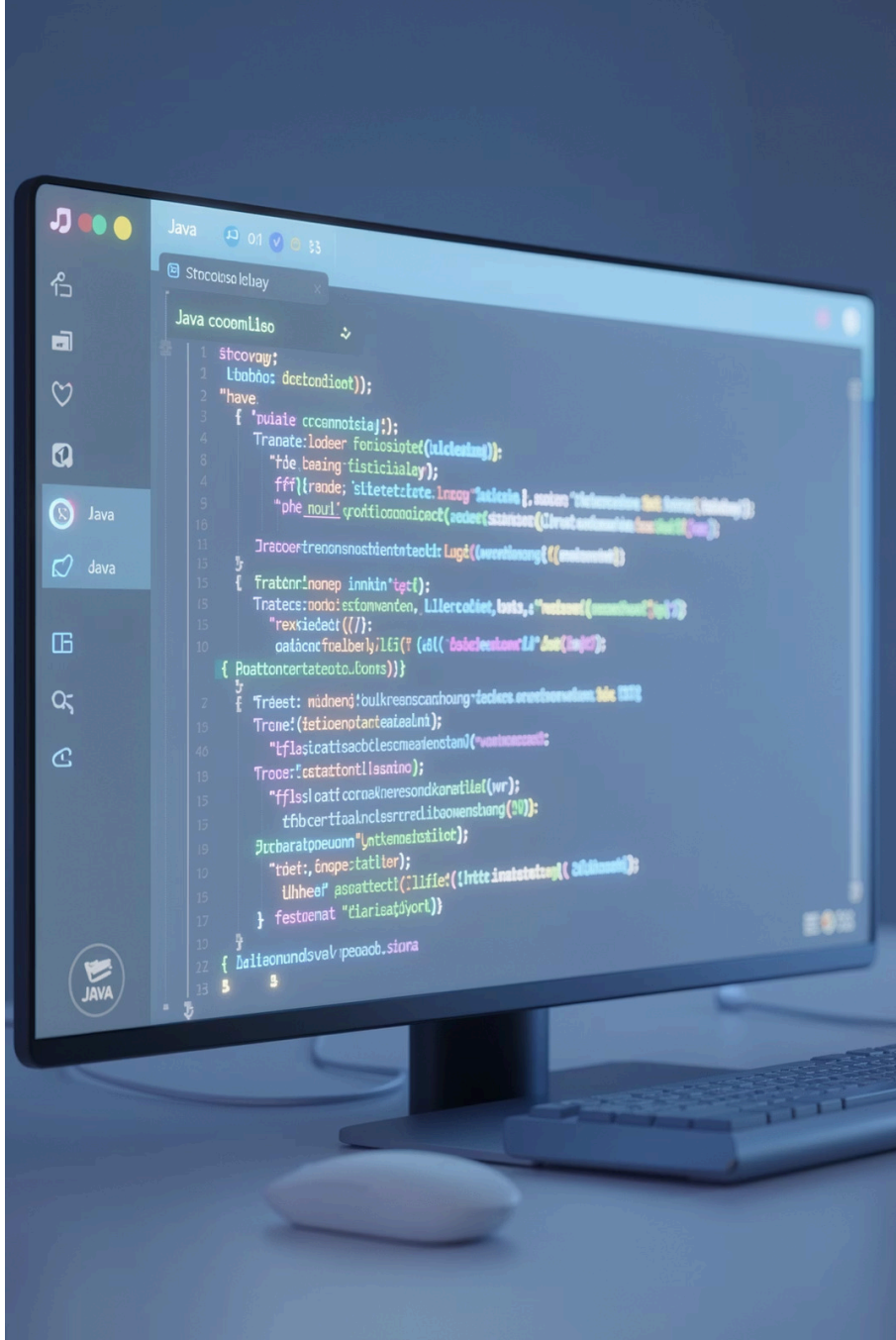
La combinación estándar para abrir y leer texto en Java con eficiencia.

readLine() en bucle

Se recorre el archivo completo línea a línea hasta que `readLine()` devuelve `null`.

Mostrar lo guardado

El programa recupera y muestra por consola exactamente lo que se escribió en sesiones anteriores.



MINIPROYECTO 4

Sistema de ventas en consola

> "Primer sistema funcional con persistencia"

Menu interactivo

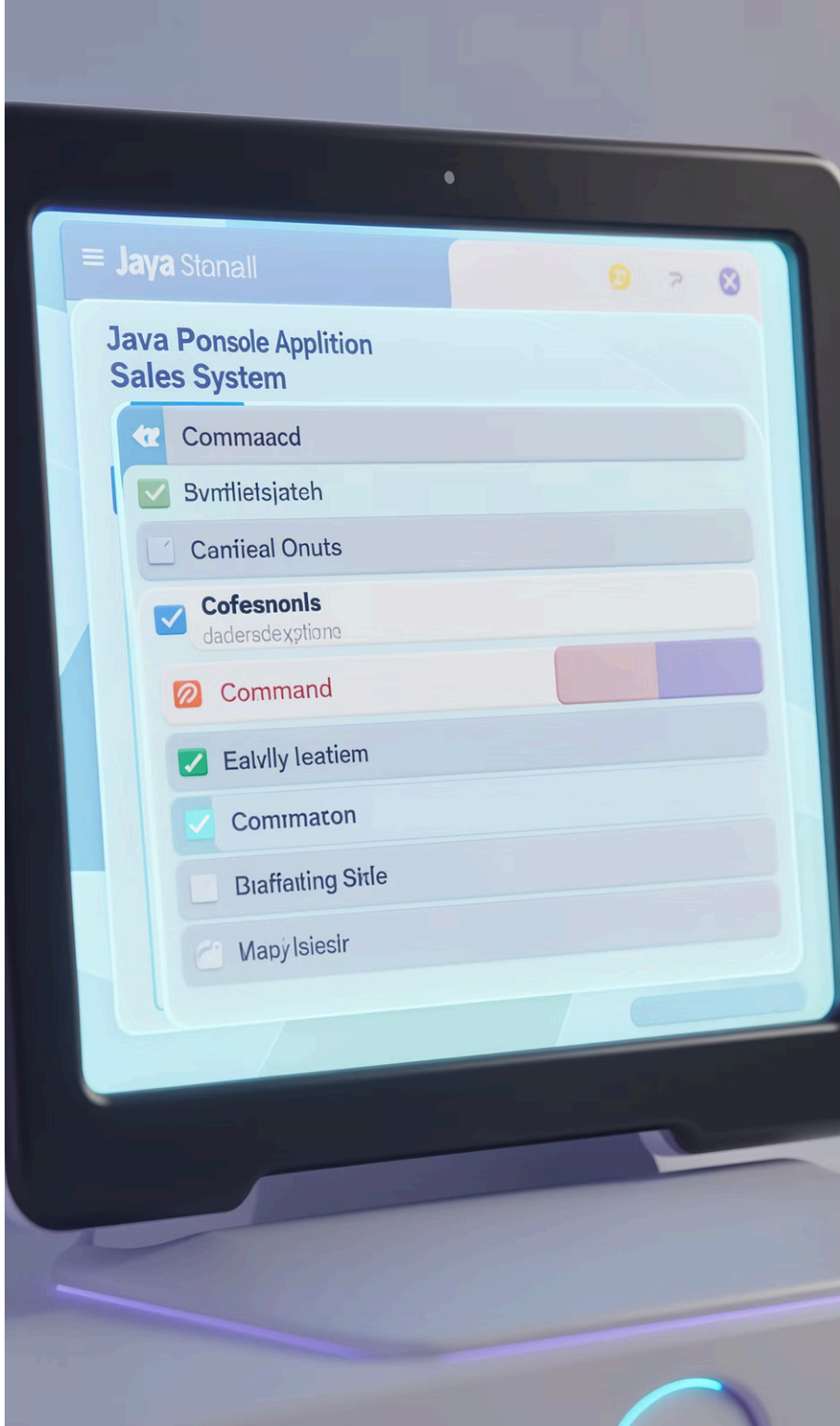
El usuario elige entre registrar una venta o consultar el historial desde el menu de consola.

Registrar ventas

Producto, cantidad y precio se guardan en el archivo con append. Cada registro ocupa una línea.

Visualizar historial

El programa lee el archivo y muestra todas las ventas registradas, de cualquier sesion anterior.



MINIPROYECTO 5

Manejo de excepciones básicas

> "El programa ya no explota"

Los errores son inevitables. La diferencia está en si el programa los gestiona o simplemente falla.

Entrada inválida

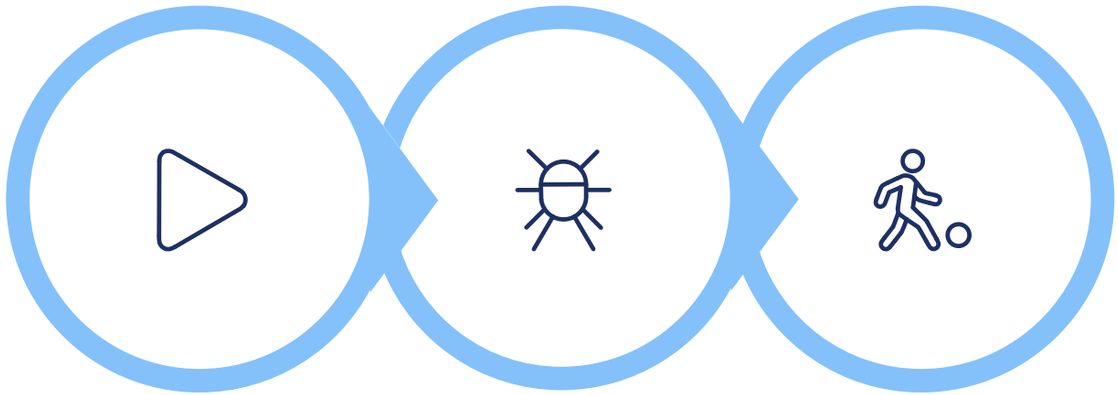
`NumberFormatException` cuando el usuario escribe texto donde se espera un número.

División por cero

`ArithmeticException` al realizar operaciones con denominador cero.

Bloque try-catch

Captura el error, muestra un mensaje claro y deja que el programa continue ejecutándose.



Try **Catch** **Continuar**

MINIPROYECTO 6

Try-with-resources

> "Java se encarga de cerrar por ti"

El problema

Si un archivo se abre y ocurre un error antes de cerrarlo, el recurso queda abierto. Esto provoca fugas de memoria y archivos bloqueados.

La solución

La sintaxis `try (FileWriter fw = ...) { }` garantiza el cierre automático del recurso al salir del bloque, con o sin error. Código más limpio y seguro.

Cualquier clase que implemente **AutoCloseable** puede usarse con try-with-resources.

MINIPROYECTO 7

Archivo inexistente: no todo error es igual

> "No todo error es igual"

1

FileNotFoundException

El archivo que se intenta leer no existe en la ruta indicada. Error específico y anticipable.

2

IOException

Error genérico de entrada/salida: disco lleno, permisos denegados, fallo de red. Cubre casos más amplios.

3

Captura diferenciada

Usando múltiples bloques `catch`, el programa responde de forma distinta según el tipo de problema.

MINIPROYECTO 8

Programa robusto con validación

> "Programa preparado para el mundo real"

1

Validar entrada

Comprobar que los datos del usuario son correctos antes de procesarlos.

2

Manejar errores

Capturar excepciones específicas y mostrar mensajes útiles al usuario.

3

Persistir datos

Guardar y recuperar información de forma segura con try-with-resources.

Qué estamos aprendiendo realmente

Más allá del código, cada miniproyecto entrena una habilidad concreta que define a un programador competente.

Persistencia de datos

Guardar información que sobrevive al cierre del programa.

Lectura de archivos

Recuperar y procesar datos almacenados en ficheros de texto.

Escritura eficiente

Usar BufferedWriter y modo append para gestionar datos de forma óptima.

Manejo de excepciones

Controlar errores predecibles sin interrumpir la ejecución del programa.

Validación de entrada

Verificar que los datos del usuario son correctos antes de usarlos.

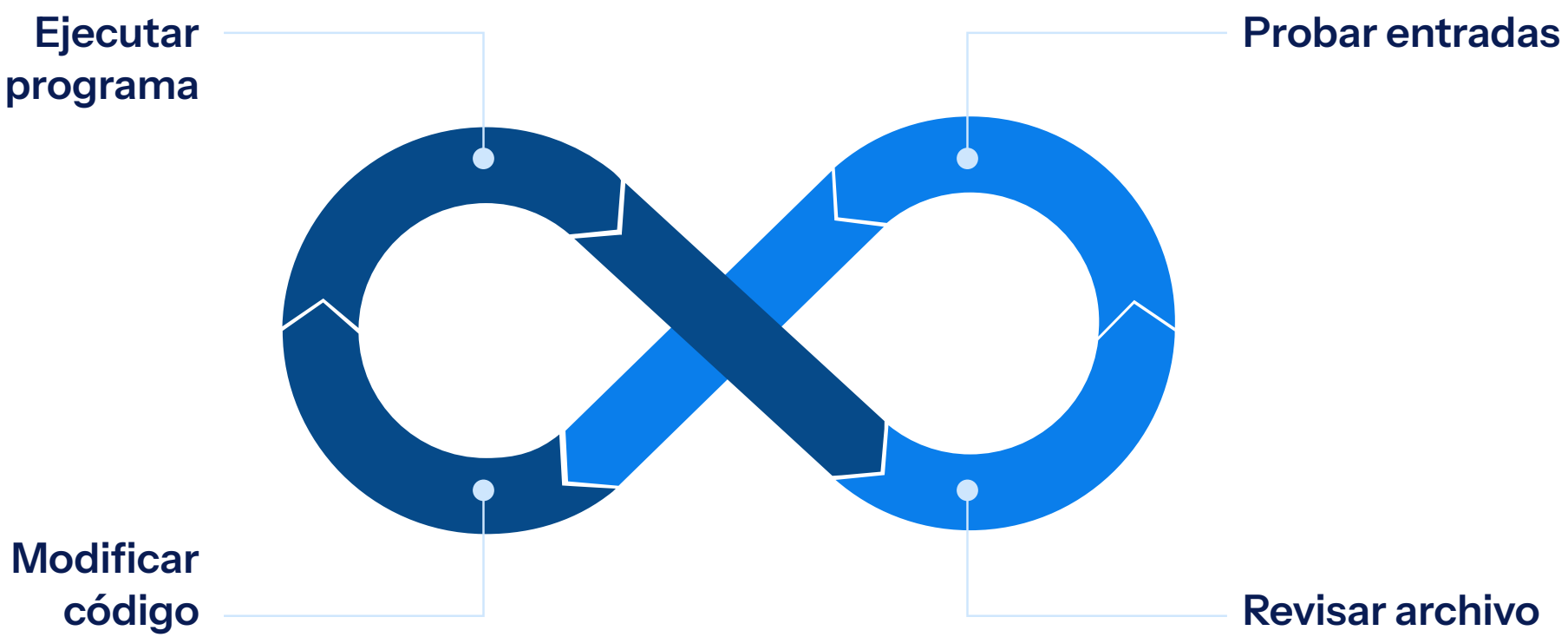
Control del flujo

Diseñar programas que respondan correctamente ante cualquier situación.

La forma de trabajo

> "Aprender haciendo, no memorizando"

El metodo es iterativo. Cada vuelta del ciclo añade comprension. No se avanza memorizando sintaxis, sino ejecutando, observando y corrigiendo.



Cada iteracion de este ciclo revela algo nuevo: un comportamiento inesperado, un error controlable o una mejora posible. Así funciona el aprendizaje real de programacion.

Cierre

De ejercicio a aplicación

> "Un programa que guarda datos y maneja errores deja de ser un ejercicio… y se convierte en una aplicación."

Escribe

Crea archivos y acumula datos con FileWriter y append.

Lee

Recupera lo guardado con BufferedReader línea por línea.

Protege

Captura excepciones y valida entradas para que nada falle.